Filtro de Qualidade do Ambiente

Guia inicial de desenvolvimento
Gabriel da Freiria · 16/10/2026 · Versão 1.0 do escopo simplificado

Objetivo

Desenvolver um projeto Java com Quarkus que receba uma imagem e disponibilize funções matemáticas de qualidade separadas. Cada função poderá ser chamada individualmente e testada. O Postman será usado para enviar imagens à API de demonstração; outras equipes poderão reutilizar as funções Java na integração de seus sistemas.

Este guia substitui o planejamento amplo anterior como roteiro de execução imediato. O escopo atual é o Filtro de Qualidade do Ambiente. Não inclui descritores de identidade, comparação facial, banco de dados, frontend ou as oito fases do projeto maior.

Tecnologias e organização

Quarkus é requisito da Celepar. Os cálculos usarão Java e operações clássicas de visão computacional; BoofCV será a referência inicial. OpenCV poderá ser incluído quando houver necessidade ou vantagem demonstrada, considerando suas dependências nativas. Não usar IA generativa nem introduzir modelos treinados neste escopo.

```
filtro-qualidade-ambiente/
  pom.xml
  README.md
  src/main/java/br/com/celepar/qualidadeimagem/
    api/
    imagem/
    resultado/
    filtros/
      unicidadefacial/
      oclusaoobjetos/
      nitidez/
      luminancia/
      simetrialuz/
      resolucaoefetiva/
      pose/
  src/main/resources/application.properties
  src/test/java/br/com/celepar/qualidadeimagem/
```

O pacote corporativo é uma sugestão a adequar à equipe. Nomes de pacotes serão minúsculos, sem espaços, acentos ou hífens. Giro da cabeça e Yaw são a mesma verificação; ficarão em pose. Criar os pacotes conforme as funções forem implementadas, sem classes vazias ou camadas redundantes.

Parte	Responsabilidade
api	Receber arquivo por HTTP, chamar a função e devolver resposta.
imagem	Decodificação, validação de arquivo e conversões compartilhadas.
resultado	Medidas, situação da avaliação e motivos.
filtros	Cálculos independentes dos detalhes HTTP.
src/test/java	Testes matemáticos e de integração com Quarkus.

Funções e critérios de avaliação

As operações abaixo seguem a tabela do documento original. Os valores são parâmetros iniciais a testar, não garantias de precisão. Registrar escala de intensidade, resolução e região analisada para que as medidas sejam comparáveis.

Função	Operação e regra de referência	Dependência
Unicidade facial	Segmentação YCbCr, componentes conectados e critérios geométricos; uma região facial candidata.	Validar com imagens sem rosto, uma face, várias faces e fundos semelhantes à pele.
Oclusão ou objetos	Fração de pele por regiões e Hough para linhas; fração de pele ≥ 0,78 como hipótese inicial.	Região facial confiável; mãos podem manter a fração de pele. Testar acessórios e barba.
Nitidez ou foco	Variância do Laplaciano; valor ≥ 100 como hipótese inicial.	Fixar escala, kernel, resolução e tratamento de bordas.
Luminância média	μ = soma das intensidades / quantidade de pixels; 60 ≤ μ ≤ 200 em escala de 0 a 255.	Definir conversão de RGB para cinza e região analisada.
Simetria de luz	Diferença absoluta entre as médias dos lados; diferença menor que 15.	Região facial localizada e coerentemente alinhada.
Resolução efetiva	Distância euclidiana entre pupilas; IPD ≥ 80 pixels.	Localizar pupilas e medir em coordenadas da imagem original.
Giro da cabeça ou Yaw	min(dL,dR)/max(dL,dR) ≥ 0,85; distâncias horizontais do nariz aos olhos.	Localizar olhos e nariz; tratar denominador zero. Não é garantia de ângulo 3D.

Primeira função de ponta a ponta

Começar por luminância: enviar imagem no Postman, recebê-la na API, abrir e validar o arquivo, converter para uma escala de cinza definida, calcular a média e devolver medida e avaliação. A primeira versão analisará a imagem inteira e informará isso explicitamente. Avaliação do rosto dependerá de sua localização posterior.

A função de cálculo receberá dados de imagem e parâmetros; não conhecerá Postman ou requisições HTTP. Cada resposta identificará medida, unidade/escala, região, limite aplicado e resultado. Diferenciar entrada inválida, aprovado, reprovado, inconclusivo e não avaliado.

Cuidados de interpretação

Arquivo decodificável não prova que seja uma fotografia. Uma candidata encontrada por cor e geometria não confirma sozinha um rosto. Essas funções não identificam pessoas nem comprovam presença física. Falta de pontos necessários deve gerar resultado inconclusivo, nunca uma medida inventada.

Preservar o original durante a análise. Limitar tamanho em bytes e pixels. Calcular qualidade de entrada antes de tratamentos que possam esconder defeitos. Medir desempenho e compatibilidade nos ambientes testados, sem prometer execução em qualquer aparelho.

Ambiente Linux e sequência de trabalho

Estado já observado

No print enviado em 15/10/2026, Java e javac responderam como OpenJDK 21.0.12.1 e Git como 2.43.0, em ambiente Linux com distribuição Java Ubuntu. Não há versão Quarkus ou modelo corporativo fornecido. Instalação da extensão Java, Postman e criação do projeto ainda precisam ser confirmadas.

1. Preparar o VS Code

Abrir Extensões com Ctrl+Shift+X, pesquisar Extension Pack for Java e instalar o pacote publicado pela Microsoft, se ainda não estiver instalado. Ele oferece suporte à edição, depuração e testes Java. O JDK já identificado será aproveitado.

2. Abrir ou preparar a pasta

Usar o nome técnico filtro-qualidade-ambiente. Se já houver uma pasta com arquivos, conferir seu conteúdo antes de criar ou extrair qualquer projeto. Não apagar a estrutura anterior; o novo nome permite manter o material existente separado.

```
mkdir -p ~/projetos/filtro-qualidade-ambiente
cd ~/projetos/filtro-qualidade-ambiente
code .
```

mkdir cria pastas; -p cria intermediárias se necessário; ~ representa a pasta pessoal; cd muda a pasta atual; code . abre essa pasta no VS Code. Se code não existir no terminal, usar Arquivo → Abrir Pasta. Manter a pasta sem fontes manuais até gerar a estrutura Quarkus.

3. Preparar o Postman

Se já estiver instalado, apenas confirmar que abre. Caso contrário, obter o aplicativo desktop na página oficial abaixo, escolhendo Linux e a arquitetura correta. Consultar uname -m: x86_64 corresponde a x64 e aarch64 a ARM64. A instalação deve seguir o procedimento permitido no equipamento corporativo. A aplicação desktop será utilizada para chamadas à API local.

4. Criar o projeto e implementar gradualmente

Após confirmar editor, pasta e Postman, gerar a base Quarkus com Java 21 e Maven Wrapper, selecionando e registrando uma versão estável compatível. A API precisará de suporte REST e JSON para receber o arquivo e devolver medidas. Definir dependências exatas na criação; não instalar BoofCV ou OpenCV isoladamente agora.

Sequência: criar e construir o projeto; implementar entrada de imagem; implementar luminância; testar pelo Postman; adicionar nitidez; avançar para localização e funções dependentes. Não presumir que os filtros de localização facial terão a mesma complexidade das médias e variâncias.

Testes e documentação enxutos

Manter README com execução, funções, fórmulas, exemplos e limitações; testes automatizados para os cálculos e integração; coleção Postman para as chamadas. Na luminância, testar imagem uniforme escura, intermediária e clara, arquivo inválido e limites de entrada. Registrar esperado e observado sem chamar exemplos de testes executados. Fotos privadas e credenciais ficam fora do Git.

Próximo retorno: confirmar extensão Java instalada, pasta aberta e se o Postman já está disponível. Nenhum código ou teste do filtro foi executado até a emissão deste guia.

Fontes oficiais consultadas em 16/10/2026:

https://code.visualstudio.com/docs/languages/java

https://www.postman.com/downloads/

https://learning.postman.com/docs/getting-started/installation/install-app/

Referência funcional: tabela “Etapa 1: Filtro de Qualidade do Ambiente”, fornecida por Gabriel.